

Assignment - 3

FeatureMap Visualization of a face image

```
import tensorflow as tf
```

```
from tensorflow.keras.applications import MobileNetV2
```

```
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.preprocessing import image
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
img-path = '/home/saidul/Desktop/saidul-0.jpg'
```

```
img = image.load_img(img-path, target_size = (224, 224))
```

```
x = image.img_to_array(img)
```

```
x = np.expand_dims(x, axis=0)
```

```
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
```

```
base_model = MobileNetV2(weights='imagenet', include_top  
= False)
```

```
layer_name = ['conv1', 'block-2-expand', 'block-6-expand',  
              'block-13-expand']
```

```
outputs = [base_model.get_layer(name).output for name in  
            layer_name]
```



```
vis_model = Model(inputs=base_model.input, outputs=output)
```

```
activations = vis_model.predict(x)
```

```
def plot_feature_maps(activations, layer_name, n_cols=8):
```

```
    n_features = activations.shape[-1]
```

```
    n_rows = n_features // n_cols + 1
```

```
    plt.figure(figsize=(16, n_rows*2))
```

```
    for i in range(n_features):
```

```
        plt.subplot(n_rows, n_cols, i+1)
```

```
        plt.imshow(activations[0, :, :, i], cmap='viridis')
```

```
        plt.axis('off')
```

```
    plt.suptitle(f'Feature Maps - {layer_name}')  
    plt.tight_layout()
```

```
    plt.show()
```

```
for i, act in enumerate(activations):
```

```
    plot_feature_maps(act, layer_names[i])
```


Analysis

The feature maps extracted from a pre-trained MobileNet V2 model after processing my own face image clearly demonstrate the hierarchical learning capability of convolutional neural networks. In the early layers, the activations primarily highlight low-level features such as edges, contours and basic textures of the face. As we move to the middle layers, the feature maps become more abstract, capturing mid-level patterns like facial parts and skin textures. In the deeper layers, the activations focus on high level semantic features, responding strongly to the overall face structure and identity-related characteristics. This progression from simple edge detection to complex object representations validates the effectiveness of deep CNNs in building increasingly abstract and task-relevant features through successive convolutional layers.

#Custom CNN:

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D,
MaxPooling2D, Dropout, Flatten, Dense, Batch Normalization

def create_custom_cnn(activation='relu'):

model = Sequential([

Conv2D(32(3,3), padding='same', activation =
activation, input_shape=(32, 32, 3)),

Batch Normalization(),

Conv2D(32, (3,3), padding='same', activation=
activation),

Batch Normalization(),

Max Pooling2D((2,2)),

Dropout(0.25),

Conv2D(64, (3,3), padding='same'),

Batch Normalization(),

~~Conv~~

Max Pooling2D((2,2)),

Dropout(0.25),

Conv2D(128, (3, 3), padding='same'),

Batch Normalization(),

Max Pooling 2D((2, 2)),

Dropout(0.25),

Flatten(),

Dense(256, activation='activation'),

Batch Normalization(),

Dropout(0.5),

Dense(10, activation='softmax')

])

return model

import tensorflow as tf

from tensorflow.keras.datasets import cifar10

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train, x_test = x_train / 255.0, x_test / 255.0

activation = ['relu', 'sigmoid', 'softmax']

histories = {}

for act, in activations:

model = create_custom_cnn(activation=act)

model.compile(optimizer='adam', loss=

'sparse_categorical_crossentropy',

metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs,

batch_size=64, validation_data=(x_test, y_test),

verbose=1)

historical[act] = history

Comparative Analysis: The performance of the CNN was evaluated using three activation functions: ReLU, Sigmoid and Softmax. ReLU demonstrated the best overall performance, achieving the highest validation accuracy of 79.17% after only 3 epochs. Sigmoid showed moderate performance with a

validation accuracy of 65.62% but suffered from slower convergence and slightly higher loss values, which is expected due to its tendency to cause vanishing gradients in deeper networks. Softmax activation performed comparably to ReLU in later epochs, reaching 73.24% validation accuracy, though it exhibited more unstable behavior in the first epoch. These results clearly indicate that ReLU is the most effective activation for this architecture on CIFAR-10 as it promotes faster learning and better gradient flow compared to Sigmoid, while Softmax is not suitable despite its competitive final accuracy.

Convolutional Kernel Type: The choice of convolutional kernel significantly influences a CNN's efficiency.

Regular kernels, typically 3×3 or 5×5 provide straightforward feature extraction but are computationally

expensive due to high parameter count. Dilated kernels increase the receptive field without losing spatial resolution by introducing gaps, making them effective for capturing broader context in tasks like semantic segmentation, though they may lose fine local details. Deformable kernels enhance flexibility by learning spatial offsets, allowing the model to adapt to geometric variations and deformations, which is particularly useful for irregular objects. Depthwise separable kernels, popularized by MobileNet, reduce computational cost by separating spatial and channelwise operations, offering an excellent trade-off. Modified depthwise-separable kernels further improve upon this by incorporating residual connections or

additional operation for better gradient flow. Finally, pointwise kernels efficiently combine information across channels with minimal computation and are often used in conjunction with depthwise convolution.

Overall, while regular kernels remain simple and effective, modern variants like depthwise separable and dilated kernels are preferred in contemporary architectures for balancing accuracy, speed and efficiency.


```
In [ ]: import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt
```

```
In [3]: # Load one face images
img_path = '/home/saidul/Desktop/4-2/Deep Learning/Assignment1/data/faces/pe
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
```

```
In [4]: # Load pre-trained model MobileNetV2
base_model = MobileNetV2(weights='imagenet', include_top=False)

# layers for MobileNetV2 to visualize (early, middle, deep)
layer_names = [
    'Conv1',          # Early: Initial convolution
    'block_3_expand', # Middle: Expansion layer in block 3
    'block_6_expand', # Deep: Expansion layer in block 6
    'block_13_expand' # Very Deep: Expansion layer in block 13
]

outputs = [base_model.get_layer(name).output for name in layer_names]
vis_model = Model(inputs=base_model.input, outputs=outputs)

# Get activations
activations = vis_model.predict(x)
```

/tmp/ipykernel_38004/4013966172.py:2: UserWarning: `input\_shape` is undefined or non-square, or `rows` is not in [96, 128, 160, 192, 224]. Weights for input shape (224, 224) will be loaded as the default.

```
base_model = MobileNetV2(weights='imagenet', include_top=False)
1/1 ————— 1s 1s/step
```

```
In [5]: # Visualize
def plot_feature_maps(activations, layer_name, n_cols=8):
    n_features = activations.shape[-1]
    n_rows = n_features // n_cols + 1
    plt.figure(figsize=(16, n_rows*2))
    for i in range(n_features):
        plt.subplot(n_rows, n_cols, i+1)
        plt.imshow(activations[0, :, :, i], cmap='viridis')
        plt.axis('off')
    plt.suptitle(f'Feature Maps - {layer_name}', fontsize=14)
    plt.tight_layout()
    plt.show()

for i, act in enumerate(activations):
    plot_feature_maps(act, layer_names[i])
```


Feature Maps - Conv1





















